

Themes in song lyrics across decades

Introduction

When trying to choose a topic for this final project and aligning with my personal interest. I've always been fascinated by music literature, the way that written lyrics can convey emotions and represent current societal issues. So I wanted to choose a topic that focuses on the analysis of songs. Music is a multisensory system experiences, but considering the difficulty of analyzing visual music video or auditory melody, I've decided to focus on song lyrics for my final project.

The goal is to identify the recurring themes and evaluate for differences or similarity in the number one hit song across decades. Though I am doing my bachelor's in computer science, I will admit that coding is not my strongest suit and therefore did not want to overcomplicate it. So my idea is to use natural language processing (NLP) techniques such as topic modeling and semantic interpretation to identify the themes and analyze the lyrics.

- Research Questions
 1. *"How do themes in popular song lyrics from the 70s, 80s, and 90s reflect emotions?"*
 2. *"What recurring themes dominate hit songs from different decades, and how do they compare?"*

Data Collection and preprocessing

To narrow the scope of analysis, I chose the hit songs from 1970s, 80s and 90s respectively. Due to problems such as lack of available archived data and charts from Billboard, as I originally plan to extract data from that. I decided chose five widely acclaimed songs from each decades given some research from this [Kaggle source](#), I've selected 5 different artist in each time frame and proceed to selected their biggest hit song for analysis.

:

1970s

1. *"Stairways to Heaven" by Led Zeppelin (1971)*
2. *"Rocket man" by Elton John (1972)*
3. *"Us and Them" by Pink Floyd (1973)*
4. *"Bohemian Rhapsody" by Queen (1975)*
5. *"Dreams" by Fleetwood Mac (1977)*

1980s

1. *"Thriller" by Michael Jackson (1982)*
2. *"Purple Rain" by Prince (1984)*
3. *"Like a Virgin" by Madonna (1984)*
4. *"Running Up That Hill" by Kate Bush (1985)*
5. *"With or Without You" by U2 (1987)*

1990s

1. *"End of the Road" by Boyz II Men (1992)*
2. *"I Will Always Love You" by Whitney Houston (1992)*
3. *"That's the Way Love Goes" by Janet Jackson (1993)*
4. *"All I Want for Christmas Is You" by Mariah Carey (1994)*
5. *"No Scrubs" by TLC (1999)*

The next step, I extracted the lyrics from Genius, an open-source platform, and saved them in txt. file. Given the variety in time period, artist, song genre and especially popularity, I would consider them representative of their respective decades in terms of impact, as these are all hit songs that the general public can agree on.

Then proceeding to preprocessing of the raw data, I've taken measures to remove common filler and stopwords from the lyrics, lowercase the text and remove punctuations and special characters. This is done through the help of code provided from [GeeksforGeeks](https://www.geeksforgeeks.org/python-regular-expressions/). I then added some custom and more refined stopwords myself, such as "ohh, woo, mm", that are commonly presented in lyrics. In which does not add much content to the meaning of lyrics.

In the code shown below, we first import stopwords list from *nltk* library, and after creating some custom stopwords, I join the stopwords together in one set. Then proceed to breakdown texts into individual words. Then filtering out the words from stopwords list, we get a list of words in “x” format. Next step is to lowercase these words and then to remove punctuations. Last step is to join the word into one string (i.e., text format again).

```
1 import nltk
2 nltk.download()
3
4 from nltk.corpus import stopwords
5 nltk.download('stopwords')
6
7 nltkstopwords = set(stopwords.words('english'))
8
9 extraword = nltkstopwords.union({'oh','ooh','mm','hmm','ah','wooo','gon','na'})
10
11 print(extraword)
12
13 from nltk.tokenize import word_tokenize
14
15 lyrics = """She packed my bags last night, pre-flight
16 Zero hour: 9:00 AM
```

```
17 And I think it's gonna be a long, long time"""
18
19
20 tokens = word_tokenize(lyrics)
21
22 filtered = [w for w in tokens if not w.lower() in extraword]
23
24 filtered = []
25
26 for w in tokens:
27     if w not in extraword:
28         filtered.append(w)
29
30 print(tokens)
31 print(filtered)
32
33 import string
34
35 filtered = [w.lower() for w in tokens if w.isalpha() and w.lower() not in extraword]
36
37 print("Filtered Words:")
38 print(filtered)
39
40 cleaned_text = " ".join(filtered)
41
42 print("Cleaned text:")
43 print(cleaned_text)
```

(The code showcases the example of when cleaning the lyrics for “rocket man”. Part of the lyrics has been cropped out when screenshotting the VS code page)

The processed and cleaned text were copied from terminal to a folder all under txt.file. This allows us to proceed to the next step—identifying the themes in the processed texts respectively.

Method

LDA & LSA Model

Initially, I tried to employ Latent Dirichlet Allocation (LDA) and [Latent Semantic Analysis](#) (LSA) tool to analyze for the themes in each song's lyrics. This approach allows us to identify clusters of words that frequently occur together, which are then interpreted as themes. This step is also done in python, through the use of *sklearn* library and *gensim*.

I've considered inputting the raw lyrics data into this model and tried to pre-run it, also the formatted data but without the removal of stopword. However it is difficult to analyze those text, it was hard for the model to give a precise prediction as well.

For LDA, the cleaned text of each songs is first vectorized, then the lyrics were stored in document that are loaded later. The LDA model is implemented and trained the data. The topic was limited to one for each century and then printed and displayed in the terminal as follows:

```

1 import os
2 from sklearn.feature_extraction.text import CountVectorizer
3 from sklearn.decomposition import LatentDirichletAllocation
4
5 path = "/Users/jcmac/Downloads/lyrics-analysis/cleaned-lyrics/70s"
6
7 documents = []
8 filenames = []
9 for filename in os.listdir(path):
10     if filename.endswith(".txt"):
11         with open(os.path.join(path, filename), 'r') as file:
12             documents.append(file.read())
13             filenames.append(filename)
14
15 print(documents)
16
17 def display_topics(model, feature, topwords):
18     for idx, topic in enumerate(model.components_):
19         print(f" Topic {idx + 1}:")
20         print(", ".join([feature[i] for i in topic.argsort()[: -topwords - 1: -1]]))
21         print("\n")
22
23
24 vectorizer = CountVectorizer(max_features=1000, stop_words='english')
25 X = vectorizer.fit_transform(documents)
26
27 lda = LatentDirichletAllocation(n_components=1, random_state=42)
28 lda.fit(X)
29
30 print("Common theme by centuries:")
31 display_topics(lda, vectorizer.get_feature_names_out(), 10)
32 |

```

(Example of code for 70s lyrics analysis)

Common theme by centuries:

Topic 1:

long, think, time, man, know, let, rocket, round, really, come

Similarly for LSA, different model were used, however the results remained similar as it identified the most common words used with its frequency.

```

1 import os
2 from gensim import corpora, models
3 from gensim.parsing.preprocessing import preprocess_string, STOPWORDS
4 from gensim.utils import simple_preprocess
5
6 path = "/Users/jcmac/Downloads/lyrics-analysis/cleaned-lyrics/70s"
7
8 documents = []
9 filenames = []
10 for filename in os.listdir(path):
11     if filename.endswith(".txt"):
12         with open(os.path.join(path, filename), 'r') as file:
13             content = file.read()
14             documents.append(content)
15             filenames.append(filename)
16
17 print(f"Loaded {len(documents)} documents.")
18
19 def preprocess(text):
20     return [token for token in simple_preprocess(text) if token not in STOPWORDS]
21
22 preprocessed = [preprocess(doc) for doc in documents]
23
24 dictionary = corpora.Dictionary(preprocessed)
25 corpus = [[dictionary.doc2bow(doc) for doc in preprocessed]]
26
27 topics = 1
28 lsamodel = models.LsiModel(corpus, id2word=dictionary, num_topics=topics)
29
30 print("Common Theme by century:")
31 topics = lsamodel.print_topics(num_words=10)
32 for topic in topics:
33     print(topic)

```

Loaded 5 documents.

Common Theme by century(70s):

(0, '0.657"long" + 0.433*"think" + 0.359*"man" + 0.336*"time" + 0.252*"rocket" + 0.101*"fuse" + 0.101*"touchdown" + 0.101*"home" + 0.101*"burning" + 0.101*"brings")*

Although this provided some insights, the themes that were attempted at identifying were often vague and generic, dominated by frequently repeated words in the lyrics rather than its actual conceptual meanings.

Semantic interpreter

Which lead me to the attempt of semantic interpreter tool. To improve the accuracy of theme identification, I utilized a pre-trained model from Hugging Face transformers. I searched in Hugging face for natural language processing model, and decided to employ a zero shot

classification tool called [facebook/bart-large-mnli](https://huggingface.co/facebook/bart-large-mnli). As zero shot classification is when *a model is trained on a set of labeled examples but is then able to classify new examples from previously unseen classes*(<https://huggingface.co/tasks/zero-shot-classification>). This specific model chosen can essentially classify text into pre-defined themes without requiring additional training.

I proceeded by deciding on some common themes presented in lyrics/literature work, as follows: “Love”, “Freedom”, “Dreams”, “Faith”, “Existential”, “Loneliness”. And I tried to combine the lyrics from one century together and identify a more combined theme.

```
1 import os
2 from transformers import pipeline
3
4 classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli", device=0)
5
6 path = "/Users/jcmac/Downloads/lyrics-analysis/cleaned-lyrics/70s"
7
8 alllyrics = ""
9 for filename in os.listdir(path):
10     if filename.endswith(".txt"):
11         with open(os.path.join(path, filename), 'r') as file:
12             alllyrics += file.read() + "\n"
13
14 result = classifier(alllyrics, ["Love", "Freedom", "Dreams", "Faith", "Existential", "Loneliness"], multi_label=True)
15 print("Combined Analysis:")
16 for label, score in zip(result["labels"], result["scores"]):
17     print(f"Theme: {label}, Confidence: {score:.2f}")
```

In which the terminal outputs for 70s:

Combined Analysis:

Theme: Freedom, Confidence: 0.99

Theme: Dreams, Confidence: 0.98

Theme: Loneliness, Confidence: 0.96

Theme: Existential, Confidence: 0.86

Theme: Love, Confidence: 0.85

Theme: Faith, Confidence: 0.85

1980s:

Theme: Love, Confidence: 0.97

Theme: Dreams, Confidence: 0.94

Theme: Faith, Confidence: 0.93

Theme: Loneliness, Confidence: 0.92

Theme: Existential, Confidence: 0.92

Theme: Freedom, Confidence: 0.81

1990s:

Theme: Existential, Confidence: 0.99

Theme: Dreams, Confidence: 0.99

Theme: Love, Confidence: 0.99

Theme: Freedom, Confidence: 0.99

Theme: Faith, Confidence: 0.99

Theme: Loneliness, Confidence: 0.99

These results were more aligned with the goal from research question and captured the essence of each song more effectively compared to the LDA/LSA techniques. However, it has its own limitations as a pre-trained model.

Analysis

Combining the results from both methods implemented in the project, pros and cons appeared. Topic modelling extracted the most common words presented in the lyrics with frequency. This offers insight into the clear prominence on words, and allowed it to be interpreted individually and straightforwardly. It is fully objective by its nature as well. However as mentioned briefly before, it fails to capture the conceptual themes and the results are rather inaccurate in identifying a deeper meaning into the lyrics. In contrast, the hugging face transformer achieved the goal more, it is more effective in extracting actual conceptual themes, and the accuracy are high given the minimal processing power. However due to its reliance on pre-trained models, there is limitation in flexibility and transparency. Resulting in difficulty in for example, modifying and finetuning.

Another observation that can be made from the result is that, all the confidence level of themes in 1990s lyrics appeared to be consistently 0.99. This indicates several issues and limitations with the model's performance, for example, inherent bias or overfitting in the model. As there were a

lot of repetitive chorus/words in the data. This could be caused partially by the lacking in range and diversity of emotions and themes as well, as the lyrics don't provide enough insights and contents for the model to perform. Or as mentioned before, a general limitation in the model as it is pre-trained already and does not allow much area for personal modification.

Referring back to the original research question, I would say the model is effective or at least able to reflect emotions in popular songs from the chosen centuries. Though this can be improved by adding more specific or wider range of pre-defined themes, however I failed to do so due to computational and processing constraints. The results also suggested consistent and high confidence level of themes across all decades(all >0.80), which could be caused by over-generalization when picking the pre-defined themes as well. Which made comparing across centuries challenging, as it is hard to capture one or two dominate themes from different decades.

Some other limitations included the structural bias in song lyrics. The nature of song lyrics structure causes certain words to disproportionately dominate the results. For example, words used in chors as it repeated significantly more compared to other parts of lyrics, such as bridges. This affects the overall judgement and can negatively impact the results generated from the model as well. The limitation of a dataset, consisting of only five songs per decades restricts the generalizability of insights and learnability of models as well. This is partially due to my choice of pre-processing method, in which the original lyrics are cleaned manually one by one. Therefore, with a biggest dataset, it would be time consuming.

Improvements

A broader and more representative dataset, for example with more songs from each decade, could provide a better insight. And this could be implemented through the implementation of better pre-processing techniques to for example, automate data cleaning and therefore expand the dataset.

We can train and fine-tune the Hugging Face transformer or with the help of it, design a more relevant and accurate lyrics and theme extraction model. And further perfect the selecting of pre-defined themes to increase diversity so that a comparative analysis across the songs from each decade can be incorporated in the future.